

实验项目	实验四 基于生成对抗网络的图像生成算法实验		
实验时间	2026.5.25	实验地点	C502
一、实验目的 通过本实验使学生熟悉深度生成网络的基本原理；掌握生成对抗网络的结构；熟练生成对抗网络构建方法和训练过程，掌握判别器和生成器的网络结构和作用。 二、实验环境 Python 编程环境。 三、实验内容 使用生成对抗网络实现手写数字图像生成，即使用真实的 MNIST 数据集，生成逼真的新的手写数字图像，主要包括数据加载与预处理、构建网络、编译网络、训练网络、性能评估和模型预测等步骤。 三、实验步骤与结果 (1)导入相关库； ①实验代码 <pre>import os os.environ['KMP_DUPLICATE_LIB_OK'] = 'True' import torch from torch import nn from torch.autograd import Variable import torchvision.transforms as tfs from torch.utils.data import DataLoader, sampler from torchvision.datasets import MNIST import numpy as np import matplotlib.pyplot as plt import matplotlib.gridspec as gridspec</pre> (2) 数据加载和预处理：从本地加载 MINIST 数据集，进行取样作为真实数据，并绘制出来。主要包含绘制图像模块和数据取样模块。 ①定义画图工具 <pre>def show_images(images): # 定义画图工具</pre>			

```

images = np.reshape(images, [images.shape[0], -1])
sqrtn = int(np.ceil(np.sqrt(images.shape[0])))
sqrting = int(np.ceil(np.sqrt(images.shape[1])))
fig = plt.figure(figsize=(sqrtn, sqrtn))
gs = gridspec.GridSpec(sqrtn, sqrtn)
gs.update(wspace=0.05, hspace=0.05)
for i, img in enumerate(images):
    ax = plt.subplot(gs[i])
    plt.axis('off')
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    ax.set_aspect('equal')
    plt.imshow(img.reshape((sqrting, sqrting)))

return

def preprocess_img(x):
    x = tf.nn.ToTensor()(x)
    return (x - 0.5) / 0.5

def deprocess_img(x):
    return (x + 1.0) / 2.0

② 定义一个取样的函数，并显示其结果。

class ChunkSampler(sampler.Sampler):
    def __init__(self, num_samples, start=0):
        self.num_samples = num_samples
        self.start = start

    def __iter__(self):
        return iter(range(self.start, self.start + self.num_samples))

    def __len__(self):
        return self.num_samples

NUM_TRAIN = 50000
NUM_VAL = 5000

```

```

NOISE_DIM = 96

batch_size = 128

train_set = MNIST('./', train=True, download=False, transform=preprocess_img)
train_data = DataLoader(train_set, batch_size=batch_size,
sampler=ChunkSampler(NUM_TRAIN, 0))

val_set = MNIST('./', train=True, download=False, transform=preprocess_img)
val_data = DataLoader(val_set, batch_size=batch_size, sampler=ChunkSampler(NUM_VAL,
NUM_TRAIN))

# 可视化图片效果

train_data_iter = iter(train_data) # 创建迭代器

batch_imgs, batch_labels = next(train_data_iter) # 获取下一批数据

# 处理图片

imgs = deprocess_img(batch_imgs.view(batch_size, 784)).numpy().squeeze()

show_images(imgs)

```

(3)构建网络：构建生成对抗网络，先定义判别器和生成器，然后将其连接起来构建生成对抗网络。

①定义判别器：包括卷积层、池化层和 Leaky ReLU 激活函数。判别器的输入包括真实的 MNIST 数据图片和生成器生成的图片。

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
class build_dc_classifier(nn.Module):
```

```
    def __init__(self):
```

```
        super(build_dc_classifier, self).__init__()
```

```
        self.conv = nn.Sequential(
```

```
            nn.Conv2d(1, 32, 5, 1),
```

```
            nn.LeakyReLU(0.01),
```

```
            nn.MaxPool2d(2, 2),
```

```
            nn.Conv2d(32, 64, 5, 1),
```

```
            nn.LeakyReLU(0.01),
```

```
            nn.MaxPool2d(2, 2)
```

```
        )
```

```

        self.fc = nn.Sequential(
            nn.Linear(1024, 1024),
            nn.LeakyReLU(0.01),
            nn.Linear(1024, 1)
        )

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.shape[0], -1)
        x = self.fc(x)

        return x

```

② 定义生成器：包括 BatchNorm1d、激活函数和 ConvTranspose2d,其中激活函数采用 ReLU 函数，输入是一组噪声信息 noise_dim。

```

class build_dc_generator(nn.Module):
    def __init__(self, noise_dim=NOISE_DIM):
        super(build_dc_generator, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(noise_dim, 1024),
            nn.ReLU(True),
            nn.BatchNorm1d(1024),
            nn.Linear(1024, 7 * 7 * 128),
            nn.ReLU(True),
            nn.BatchNorm1d(7 * 7 * 128)
        )
        self.conv = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 4, 2, padding=1),
            nn.ReLU(True),
            nn.BatchNorm2d(64),
            nn.ConvTranspose2d(64, 1, 4, 2, padding=1),
            nn.Tanh()
        )

```

③ 定义损失函数，简要说明定义了哪种基本的损失函数？

```
def discriminator_loss(logits_real, logits_fake): # 判别器的损失函数
```

```
def generator_loss(logits_fake): # 生成器的损失函数
```

④ 生成对抗网络构建：将生成器和判别器串联起来，并利用损失函数的作用说明生成对抗过程。

```
def train_dc_gan(D_net,
                  G_net,
                  D_optimizer,
                  G_optimizer,
                  discriminator_loss,
                  generator_loss,
                  show_every=250,
                  noise_size=96,
                  num_epochs=10):
    iter_count = 0
```

```

for epoch in range(num_epochs):
    for x, _ in train_data:
        bs = x.shape[0]
        real_data = Variable(x).to(device) # 真实数据
        logits_real = D_net(real_data) # 判别网络得分
        # -1~1 的分布
        sample_noise = (torch.rand(bs, noise_size) - 0.5) / 0.5
        g_fake_seed = Variable(sample_noise).to(device)
        fake_images = G_net(g_fake_seed) # 生成的数据
        logits_fake = D_net(fake_images) # 判别网络得分
        # 判别器的损失函数
        d_total_error = discriminator_loss(logits_real, logits_fake)
        D_optimizer.zero_grad()
        d_total_error.backward()
        D_optimizer.step() # 优化判别网络
        # 生成网络
        g_fake_seed = Variable(sample_noise).to(device)
        fake_images = G_net(g_fake_seed) # 生成的假的数据
        gen_logits_fake = D_net(fake_images)
        g_error = generator_loss(gen_logits_fake) # 生成器的损失函数
        G_optimizer.zero_grad()
        g_error.backward()
        G_optimizer.step() # 优化生成网络
        if (iter_count % show_every == 0):
            print('Iter: {}, D: {:.4}, G: {:.4}'.format(iter_count,
                d_total_error.item(), g_error.item()))
            imgs_numpy = deprocess_img(fake_images.data.cpu().numpy())
            show_images(imgs_numpy[0:16])
            plt.show()
            print()

```

```
iter_count += 1
```

⑤ 定义优化器：使用 Adam 优化器进行训练，学习率设为 $3e-4$, beta 参数设为(0.5,0.999)。

```
def get_optimizer(net):
```

```
    optimizer = torch.optim.Adam(net.parameters(), lr=3e-4, betas=(0.5, 0.999))
```

```
    return optimizer
```

(4)训练网络，并保存结果，结果应该有一个从不好到好的变化过程。

①实验代码

```
D_DC = build_dc_classifier().to(device)
```

```
G_DC = build_dc_generator().to(device)
```

```
D_DC_optim = get_optimizer(D_DC) # 判别器创建优化器
```

```
G_DC_optim = get_optimizer(G_DC) # 生成器创建优化器
```

```
train_dc_gan(D_DC, G_DC, D_DC_optim, G_DC_optim,  
             discriminator_loss, generator_loss, num_epochs=20)
```

②训练结果，展示 250,1000,3000,4000,5000,7000 次的结果，并用文字分析结果。

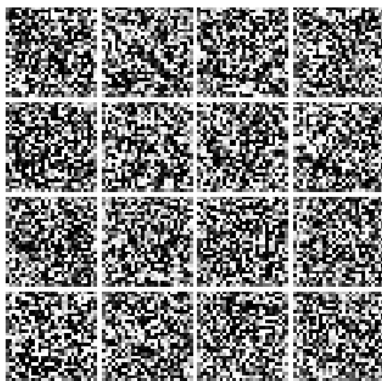


图 1-第 1 次

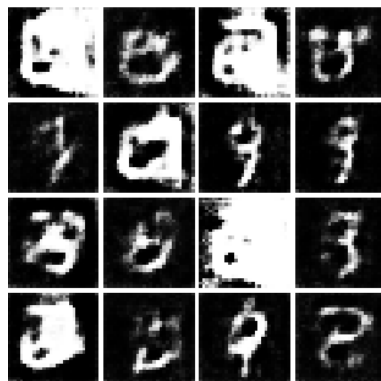


图 2-第 250 次

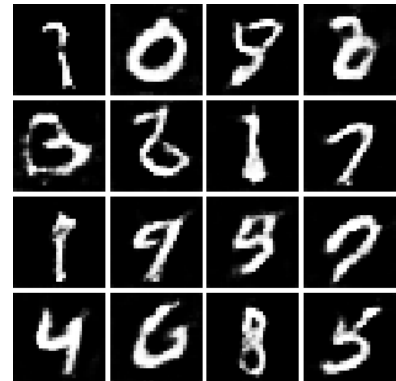


图 3-第 1000 次

图 1：第 1 次迭代

效果特征：完全是杂乱的黑白噪点，没有任何数字的轮廓和结构，和随机噪声没有区别。

图 2：第 250 次迭代

效果特征：已经出现了模糊的数字轮廓和笔画结构，能隐约看到类似 "0、3、9" 的形状，但整体非常模糊、边缘粗糙，部分数字变形严重，还有的无法辨认具体数字。

图 3：第 1000 次迭代

效果特征：生成的数字清晰、笔画完整，绝大多数可以准确辨认出是 0-9 的具体数字，和真实 MNIST 手写数字的视觉效果已经非常接近，只有少数存在轻微变形。

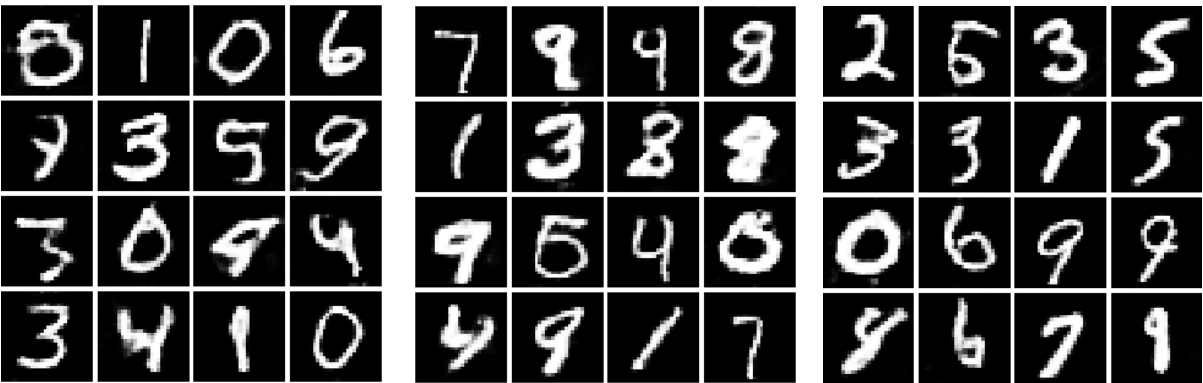


图 4-第 3000 次

图 5-第 4000 次

图 6-第 5000 次

图 4：第 3000 次迭代

生成效果特征：所有数字都清晰可辨认，0-9 的数字类别完整，没有无法识别的样本，笔画基本流畅，但存在少量小问题：部分数字有轻微变形（比如第一行的 "6"、第三行的 "4"），少数数字的笔画有轻微粘连、边缘不够平滑，数字多样性已经体现：同一个数字（比如 3、0）有不同的手写风格，没有出现严重的 "模式崩溃"（所有生成结果都一样）

图 5：第 4000 次迭代

生成效果特征：相比 3000 次，笔画更自然、边缘更平滑，变形的数字大幅减少，几乎所有数字的结构都符合真实手写习惯，数字的多样性进一步提升：比如 "9" 有不同的写法、"8" 的闭环更自然，手写风格的差异更明显，没有出现 "退化"：没有回到模糊、噪点的状态，说明训练没有崩溃

图 6：第 5000 次迭代

生成效果特征：生成质量已经接近真实 MNIST 数据集的水平：数字清晰、笔画流畅自然，几乎没有变形，普通人很难区分这是生成的假数字还是真实的手写数字，多样性达到最优：0-9 每个数字都有多种不同的手写风格，没有出现模式崩溃（没有重复的、完全一样的数字），整体一致性强：所有生成样本的质量都很高，没有 "有的清晰有的模糊" 的两极分化。

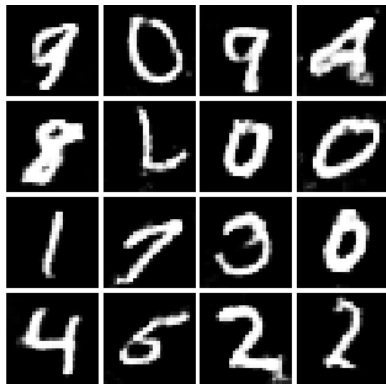


图 7-第 7000 次

图 7：第 7000 次迭代

和初期、中期相比，7000 次的生成依然保持了较高的基础质量：所有数字都清晰可辨认，没有回到噪点、模糊的崩溃状态，笔画整体流畅，灰度、对比度和真实 MNIST 数字一致，生成器的基础 "造假能力" 没有退化，但是，比 5000 次的最优结果，出现了明显的训练副作用，生成多样性下降，出现重复样本，部分数字出现 **unnatural** 变形。结论，最优训练节点是 5000 次迭代：此时生成质量、多样性、自然度同时达到峰值。

四、思考题

- (1) MNIST 原数据和生成数据有没有差异？
- (2) 简要说明生成对抗网络的网络结构？
- (3) 简述生成对抗网络损失函数，定义了几个？分别是什么？有什么作用？

实验成绩评定表

序号	考核项目	分值分布	成绩
1	预习情况	20	
2	实验任务完成情况	40	
3	实验报告质量	40	
总分			
指导教师签字			